

LeWorldModel 迁移到 Qualcomm QNN 的任务执行计划

这份文档按实际推进项目的顺序来写。整体目标是：先跑通原始 PyTorch 模型，明确 encoder 和 predictor 的输入输出，再完成 ONNX 导出、精度对齐、量化、QNN 编译、端侧部署和最终性能评估。

阶段 0：确认任务目标

先确认的问题

- 原始平台是不是 Jetson Nano。
- 目标平台是不是 Qualcomm QNN。
- 目标设备具体是什么。
- 目标设备是手机、开发板，还是其他 Snapdragon 平台。

初步判断

- 如果目标还是 Jetson Nano，应该优先走 TensorRT 路线。
- 如果目标是高通平台，才需要走 QNN 路线。
- 当前暂时按下面这个理解推进：
 - 原始模型可能是在 Jetson Nano 上跑。
 - 当前任务是吧模型迁移到高通 QNN 平台。

本阶段目的

先把任务方向确认清楚，避免后面导出、量化和部署工具链选错。

阶段 1：跑通 PyTorch baseline

执行内容

- 下载并整理原始代码。

- 配置 PyTorch 运行环境。
- 加载论文或项目提供的 checkpoint。
- 准备一批可复现的测试数据。
- 跑通原始模型推理流程。

需要记录的信息

- 模型是否能正常运行。
- 模型输入是什么。
- 模型输出是什么。
- 原始推理耗时是多少。
- 原始内存占用是多少。
- 原始精度或任务表现是多少。

待确认问题

- 是否现在先在 AutoDL 租一张 GPU 卡，把 PyTorch baseline 跑一遍并保存推理结果。

本阶段产物

- baseline_run.md
- baseline_log.txt
- baseline_latency.csv

本阶段目的

建立原始对照组。后面 ONNX、INT8、QNN 的所有结果都要和这个 baseline 对齐。

阶段 2：找到 encoder 和 predictor

需要在代码里定位的模块

- image -> latent
- latent + action -> next_latent

也就是需要找到：

- encoder.forward()

- `predictor.forward()`

需要打印和确认的 shape

- image shape
- latent shape
- action shape
- next_latent shape

待确认问题

- encoder 和 predictor 是否在官方代码里已经是独立模块，还是需要从完整模型里拆出来。
- 需要导出的是单帧 encoder，还是带历史帧输入的 encoder。
- predictor 输入的是单步 latent + action，还是一段历史 latent 和 action 序列。
- latent 的维度、batch 维度、时间维度是否要固定下来。

本阶段产物

- `model_structure.md`
- `encoder_predictor_shape.md`

本阶段目的

搞清楚到底要导出哪两个子模块，避免把训练逻辑、数据处理逻辑或规划逻辑一起错误导出。

阶段 3：封装 encoder 和 predictor

真实模型的代码可能比较复杂，不一定能直接导出。所以需要先把可部署部分单独封装出来。

encoder 封装目标

- 输入：image
- 输出：latent

predictor 封装目标

- 输入：latent, action
- 输出：next_latent

重点检查的问题

- forward 里有没有多余逻辑。
- 有没有 Python 控制流影响导出。
- 有没有动态 shape。
- 有没有 QNN 或 ONNX 不适合支持的部分。
- 是否需要把预处理或后处理放到模型外部。

本阶段目的

把模型整理成干净的、可导出的神经网络模块。

阶段 4：导出 ONNX

分别导出的模型

- encoder.onnx
- predictor.onnx

导出时需要固定的内容

- 固定输入 shape。
- 设置 input_names 。
- 设置 output_names 。
- 设置合适的 opset_version 。
- 保存一组导出时使用的 dummy input。

待确认问题

- ONNX 导出先在 AutoDL / PC 上完成，还是直接在目标部署环境附近完成。
- 输入 shape 是否全部固定，还是需要保留 batch 维动态维度。
- opset_version 选哪个版本最适合后续 QNN 工具链。
- 如果遇到不支持算子，是改模型结构，还是先保留 FP32 路线验证。

本阶段产物

- export_encoder_onnx.py

- export_predictor_onnx.py
- encoder.onnx
- predictor.onnx

本阶段目的

把 PyTorch 模型转成部署中间格式，为 ONNXRuntime 验证和 QNN 编译做准备。

阶段 5：ONNX 与 PyTorch 对齐

分别比较的结果

encoder:

- PyTorch encoder 输出的 latent
- ONNX encoder 输出的 latent

predictor:

- PyTorch predictor 输出的 next_latent
- ONNX predictor 输出的 next_latent

使用的指标

- max_diff：最大误差
- mean_diff：平均误差
- cosine similarity：余弦相似度

判断标准

- 如果误差很小，说明 ONNX 导出基本成功。
- 如果误差很大，需要回头检查导出脚本、模型模式、动态 shape、算子兼容性和输入预处理。

本阶段产物

- compare_encoder_torch_onnx.py
- compare_predictor_torch_onnx.py
- onnx_compare_report.md

本阶段目的

证明 ONNX 模型没有在导出阶段损坏。

阶段 6：做量化

优先尝试的量化方案

- INT8 PTQ 量化。

需要准备的校准数据

- encoder 校准数据：真实图像输入。
- predictor 校准数据：真实 latent 和 action。

待确认问题

- 校准数据从 baseline 推理过程中直接保存，还是单独写脚本重新生成。
- encoder 校准数据需要多少张图像，是否覆盖不同场景和动作状态。
- predictor 校准数据是否需要保存真实 latent，还是每次动态调用 encoder 生成。
- 量化优先使用 QNN 自带工具，还是先用 ONNXRuntime / 其他工具做中间验证。

量化目标

- `encoder.onnx -> encoder_int8`
- `predictor.onnx -> predictor_int8`

备选方案

如果 predictor INT8 后误差太大，再尝试：

- encoder INT8 + predictor FP16。
- 部分层 INT8，敏感层 FP16。

本阶段产物

- `calib_data/`
- `quant_encoder.py`

- quant_predictor.py
- encoder_int8.onnx 或 QNN 量化产物
- predictor_int8.onnx 或 QNN 量化产物
- quant_report.md

本阶段目的

在尽量保持模型行为不变的前提下，降低模型大小、推理延迟和端侧计算压力。

阶段 7：量化误差评估

这一阶段非常重要。不能只写量化成功了，还要证明量化后的模型没有明显损坏。

测试内容

- encoder latent 输出误差。
- predictor 单步预测误差。
- predictor 多步预测误差。
- 是否出现 NaN 。
- 是否出现 Inf 。
- 多步 rollout 是否发散。

单步预测检查

```
latent_t + action_t -> latent_{t+1}
```

多步预测检查

```
latent_0 + action_0 -> latent_1  
latent_1 + action_1 -> latent_2  
latent_2 + action_2 -> latent_3  
...
```

特别注意的问题

世界模型最怕的是：

- 单步误差看起来很小。
- 多步 rollout 后误差越滚越大。
- 最后 latent 发散，导致规划失效。

本阶段产物

- `compare_fp32_int8_encoder.py`
- `compare_fp32_int8_predictor.py`
- `multi_step_rollout_eval.py`
- `quant_error_report.md`

本阶段目的

确认 INT8 量化后的误差是否仍然在可接受范围内，尤其是多步预测是否还能稳定。

阶段 8：QNN 编译

执行内容

把 ONNX 或量化后的模型交给 QNN 工具链，生成高通设备能运行的模型产物。

编译链路

encoder:

```
encoder.onnx / encoder_int8  
-> QNN 编译  
-> QNN 模型产物
```

predictor:

```
predictor.onnx / predictor_int8  
-> QNN 编译  
-> QNN 模型产物
```


重点检查的问题

- 有没有不支持算子。
- 有没有编译 warning。
- 有没有 CPU fallback。
- 能不能跑 HTP backend。
- 编译后的输入输出名字是否和预期一致。

待确认问题

- 当前使用的 QNN SDK 版本是多少，是否和目标设备系统匹配。
- 编译目标是 HTP / DSP / CPU 中的哪一个 backend。
- INT8 模型和 FP16 / FP32 模型是否都需要编译一版作为对照。

本阶段产物

- qnn_compile_encoder.sh
- qnn_compile_predictor.sh
- qnn_compile_log.txt
- qnn_artifacts/

本阶段目的

把模型从通用中间格式真正转换为 Qualcomm 端侧可执行产物。

阶段 9：端侧部署

需要放到高通设备上的内容

- QNN runtime。
- QNN 模型产物。
- 测试输入。
- 端侧运行脚本。

端侧运行流程

- 加载 encoder。

- 运行 encoder。
- 得到 latent 。
- 加载 predictor。
- 运行 predictor。
- 得到 next_latent 。

可能的目标设备

- Snapdragon 手机。
- 高通开发板。
- Windows on Snapdragon 设备。

待确认问题

- 目标设备具体型号是什么，是否已经能连接 adb 或 ssh。
- 端侧运行环境是否已经安装 QNN runtime 依赖。
- 测试输入是直接从 baseline 保存的 .npy / .bin 文件读取，还是端侧实时采集。
- 端侧只验证 encoder / predictor 单独运行，还是要串起来跑一段完整推理。

本阶段产物

- deploy_readme.md
- run_encoder_demo.sh
- run_predictor_demo.sh
- device_run_log.txt

本阶段目的

确认模型不只是在 PC 上能导出和编译，而是真的能在目标高通设备上跑起来。

阶段 10：性能测试

比较版本

- PyTorch baseline。
- ONNX Runtime。
- QNN INT8。

记录指标

- 模型大小。
- 加载时间。
- 单次推理延迟。
- 平均推理延迟。
- 最大推理延迟。
- 内存占用。
- 功耗表现。
- 是否 crash。
- 是否存在内存泄漏。

待确认问题

- 性能测试是在 AutoDL、PC、Jetson 和高通端侧都做，还是只做 PyTorch baseline 与 QNN 端侧对比。
- 是否需要记录功耗，如果需要，使用什么工具采集。
- 任务成功率是否需要重新跑完整规划评估，还是只比较 latent 输出误差和推理速度。

结果表格模板

模型版本	平台	延迟	内存	模型大小
PyTorch FP32	Jetson / PC	xx ms	xx MB	xx MB
ONNX FP32	CPU	xx ms	xx MB	xx MB
QNN INT8	HTP / NPU	xx ms	xx MB	xx MB

本阶段目的

证明 QNN 端侧部署是否真的有价值，包括速度、内存和模型大小是否有明显收益。

阶段 11：写最终报告

最终报告需要回答的问题

- 模型能否成功导出 ONNX。

- ONNX 与 PyTorch 是否一致。
- INT8 量化是否成功。
- 量化误差是否可接受。
- QNN 编译是否成功。
- 是否跑在 HTP / NPU。
- 是否存在 CPU fallback。
- 速度有没有提升。
- 内存有没有下降。
- 任务成功率是否明显下降。
- 当前还存在哪些问题。
- 后续应该怎么优化。

最终交付物

1. ONNX 模型。
2. QNN 量化模型及编译产物。
3. ONNX 导出脚本。
4. 校准数据生成脚本。
5. 量化和编译脚本。
6. 端侧推理 demo。
7. 精度与性能测试报告。
8. 部署说明文档。
9. 已知问题和后续优化建议。

整体推进顺序

确认目标平台

- > 跑通 PyTorch baseline
- > 拆出 encoder / predictor
- > 封装可导出模块
- > 导出 ONNX
- > 对齐 PyTorch 与 ONNX
- > 做 INT8 量化
- > 评估量化误差
- > QNN 编译
- > 高通端侧运行
- > 性能测试
- > 写最终报告

后续推进时，每完成一个阶段，就把对应日志、脚本和报告补齐。这样最后写总结时，不需要重新回忆过程，只要把每个阶段的结果汇总起来即可。